

C 語言

Part I : 基本語法

Part I：基本語法

基本類型

基本類型

順帶一提，這種嚴格定義類型的語言我們習慣叫做**強型別語言**

什麼是類型？

類型是C語言裡最重要的元素之一，可以毫不猶豫地這樣說：是類型造就了C語言的偉大。

歸根究柢，什麼是類型呢？

類型是讓電腦知道如何解釋**資料**的系統。

我們用標籤來做解釋：

假設有三種不同的液體，分別裝在三種不同的容器裡，在不使用味覺的前提下，分辨這三種液體最簡單的方法就是在其容器上貼上標籤，讓大家知道這是酒精、鹽酸還是蒸餾水。

類型就好比標籤，一樣會貼在資料上面，方便電腦去分析這是一個英文字母、整數又或是小數。

基本類型

除了基本類型外，C語言也提供自定義類型，稱做複合類型

類型代號	中翻	意義
char	字元	儲存 英文字母 與 操作符號
int	整數	儲存 $-(2^{31} - 1)$ 至 $(2^{31} - 1)$ 以內的 整數
double	浮點數	儲存15至16為有效數字的小數

以上是C語言裡常見的基本類型。除了上述列出的類型，C還有像是unsigned、long long、short、float、void等等的類型。但除了short、float、unsigned與void以外，其餘的類型都可以透過「修飾」int與float得到，所以這裡就不細提了。

Part I：基本語法 資料(値)

資料(值)

什麼是值？

上一章節提到類型就像是一個標籤，被貼上標籤的就是資料，俗稱「值」。

在C語言裡面，你可以直接在程式中輸入值，像是 100、'A'、1.14514 等等。

這些值都會被C語言自動地貼上類型，分別貼上：int、char、double。

```
100 //int
'A' //char
1.14515 //double
```

BTW，這個會考！

在C語言中，以「//」標示的註解，意思是他並不是給電腦看的，而是給人看的。像是例圖這樣。

資料(值)

資料特徵	類型	範例
標準數字	int	100、032、255
被「'」包圍的字	char	'A'、'B'、'C'
帶有單「.»的標準數字	double	3.1415、1.005

以上是C語言裡常見的資料特徵。除了上述列出的三個基本特徵外，C語言還會依資料的大小來判斷是否要「修飾」這個資料的類型。

像是：原本應為int類型的數字，如果大小超過了int能負荷的程度，那C語言就會派出空間更大的long long。

Part I：基本語法

變數

變數

什麼是變數？

變數，顧名思義就是會變動的數，英文是：Variable (N.)。

在電腦科學裡，變數的變動是由我們程式設計師決定的。

像是：

第12行：x = 100; //此時x為100

第21行：x = 20; //此時x為20

變數有時也「不會變」，永遠固定為某個特定值，此時的變數我們稱為**變數的常數態**，或是稱為**常數**，英文是：Constant (N.)。

變數

為什麼需要變數？

想像一下，假設你手下有幾百名員工，你要指派他們全部人做同一件任務，你有兩種方式去通知他們：

1. 一個一個通知他們

一個一個走進你的員工的辦公室，告訴他們這次任務的目標。

2. 拿大聲公通知所有人

直接召集你的手下，並提起嗓子，指派在場的所有員工這次任務的目標。

只要你不是來整活的，一般情況下都會選擇第二種方式，不但省力，還能達成同樣的效果。

變數本身就是在做同樣的事，它讓程式設計師不用一個一個地去調整同個參數，只需要調整一個部分，就能調整全體。

變數

創建一個變數

在C語言裡，創建一個變數會需要用到三項物件：
類型、名稱、值。

類型就是我們第一章節提到的，有char、int、double。

名稱是指你所創建的變數的名稱。

值就是上個章節提到的，可以是 'A'、100、3.14。

這時創建出來的變數，它的值就是我們賦予給它的值。

這種變數，我們稱之為**初始賦予變數**。

如：

```
int x = 100; //此時x為100
```

變數

創建一個變數：初始賦予變數

<類型>

<名稱>

=

<值>

;

```
int score = 100;  
char letter = 'A';  
double pi = 3.14159;
```

變數

創建一個變數

其實在C語言裡面，也可以只透過兩個元素就創建出一個變數：類型與名稱。

透過這種方式所創造出的變數，「它沒有值」。

稱之為**空殼變數**。

例如：

```
int heigh; //heigh沒有值
```

在某些C語言的編譯器中，空殼變數可能是任何一個值而非0。

這時我們稱這個數為garbage value。garbage value可能是來自之前程式的代碼裡，因為記憶體尚未被清除的緣故，所以才會保留到當前程式中，當然這個值也有可能來自當前程式。

變數

創建一個變數：空殼變數

<類型>

<名稱>

;

```
int score;  
char letter;  
double pi;
```

Part I：基本語法

類型擴展

陣列 Part

類型擴展

類型擴展不是類型修飾喔

類型的擴展

上次，我們介紹了基本的類型，也就是`int`、`char`、`double`，分別代表整數、字元、小數。

有沒有人有發現，「字串」呢？

那個可以存放我的名子：`"Tim"`、`"Amy"` 的類型跑去哪了？

其實阿，「字串」他是一個擴展的類型，由`char`擴展而來的。因此不在基本類型的討論範圍內。所以特地創建了這個章節：類型擴展，來向大家介紹這類的類型。

類型擴展

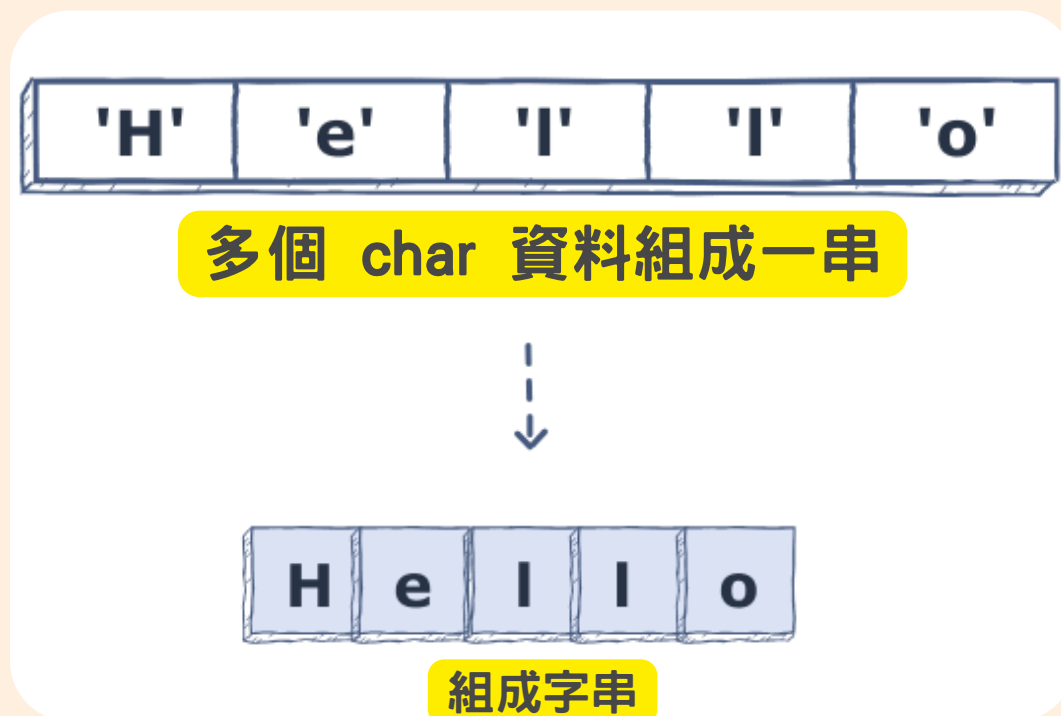
陣列

陣列，英文： Array (N.)。

在C語言中，陣列是一種類型的擴展，它可以把單一類型的多個資料組成一串。

例如：類型 **char** (字) 可以透過陣列擴展 (串) 合成字串。

以下我們以創建char類型的變數來做示範，看看類型與值都擴展成陣列長什麼樣子。



類型擴展

字元

```
char a = 'A';
```

創建char類型的變數：a，其值為 'A'。

類型：char。

值：被貼上char類型，英文字母A。

字串

```
char name[] = {'T', 'i', 'm'};
```

創建char類型的陣列變數：name，其值為 {'T','i','m'}。

類型：char陣列，AKA char[]。

值：被貼上char陣列類型，英文字串Tim。

類型擴展

陣列

觀察剛剛的程式碼差異，會發現若要進行陣列類型擴展，會需要：

1. 在變數名稱後方加上 []。
2. 值從原本單一的 'A' 變成了 {'T','i','m'}。

{ } 在這裡的意思是：寫在這括號裡的值都是陣列的一部分，每個值都會使用「,」做間隔。因此，C語言看到{ }就會知道：「喔！這個值是陣列。」以此貼上陣列的標籤，至於哪個類型的陣列，C語言則會以變數宣告的類型去做判斷。

例如：

```
char name[] = {...};
```

C語言：「我看到了 {...} ，所以這是一個陣列，嘻嘻嘻，我真聰明。接著它是誰的陣列啊？我來看看變數的類型。是char[]，所以{...}是char的陣列。」

類型擴展

字串

其實阿！字串的值可以不用寫得像天書一樣。

{'T','i','m'} 可以在C語言裡面寫成 "Tim"。使用「"」包裹起來的字詞，就是字串。

而且這樣寫也不會有守衛值(Guard value)遺失的問題。

守衛值(Guard value)是指一個字串的尾巴。

在C語言裡面，電腦為了知道字串的真實長度，會在字串的尾巴塞入守衛值，通常是0，目的在於能知道這個字串的真實長度，因此才能正常打印字串，與做字串相關的操作。

類型擴展

創建一個陣列變數：初始賦予變數

<類型>

<名稱>

[]

=

{ ...

... }

;

```
char name[] = {'T', 'i', 'm'};
```

```
int ages[] = {14, 12, 13};
```

```
double cons[] = {3.1415, 2.71828, 1.61803};
```

Part I：基本語法

輸入與輸出

輸入與輸出

printf函數名稱是由print(打印)+formatted(格式化)兩個單字組合而成的。意旨：我打印出的內容是由格式化整理而成的。

輸出

學完了類型、變數與值之後，該來實際使用這三者了。最好的使用地方就是文字打印：輸出。

在C語言中，若我們要打印一段文字，會需要用到一個名為printf的函數。我們最常遇到的printf函式大概是長這樣：

```
printf("Hello, World!");
```

根據他的長相，我們可以得知，他的工作是打印出"Hello, World!"。

輸入與輸出

輸出

可是單單輸出Hello World好像有點太單調了，我難道不能再增加點程式上的樂趣嗎？像是打印出變數？

實際上是可以的，這也是printf函數強大的地方，你可以透過字串的格式化組合，來輸出變數，無論類型是int、char、double甚至陣列。

以下我們就來講解如何輸出變數吧！

輸入與輸出

輸出

printf函數：

```
printf(格式化字串, 欲打印1, 欲打印2, ...);
```

printf：printf函數本身。

(...)：讓語言知道我們要呼叫printf函數。裡面的元素，各個都要以「,」作為間隔。

格式化字串：是一種字串，輸入特定的格式化符號，可以把該符號取代成欲打印的變數或值。

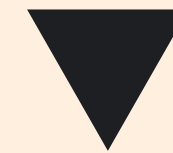
欲打印：想打印的變數或值。可以多個。

輸入與輸出

格式化符號

符號	輸出項目
%s	字串
%c	字元
%f	浮點數
%d	整數

```
printf(format: "%s", "Tim");  
printf(format: "%c", 'A');  
printf(format: "%f", 1.134);  
printf(format: "%d", 1234);
```

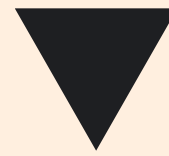


```
printf(format: "Tim");  
printf(format: "A");  
printf(format: "1.134");  
printf(format: "1234");
```

輸入與輸出

多個欲印出

```
printf(format: "Hello! My name is %s, I'm %d years old", "Timmy", 16);
```



```
printf(format: "Hello! My name is Timmy, I'm 16 years old");
```

類型擴展

scanf函數名稱也是一樣的道理。

輸入

學完了輸出，怎麼能少的瞭輸入呢？

在C語言中，若要讓使用者輸入字串或數字至程式中，會需要用到一個名為scanf的函數。

scanf與printf函數一樣，都會使用到「格式化字串」，且格式化符號所對應的類型也一樣。但是「欲打印」變成了「欲輸入」，且有幾個小地方需要做修改，我們來看看吧！

類型擴展

輸入

scanf函數：

`scanf(格式化字串, 欲輸入1, 欲輸入2, ...);`

scanf：scanf函數本身。

(...)：讓語言知道我們要呼叫scanf函數。裡面的元素，各個都要以「,」作為間隔。

格式化字串：是一種字串，輸入特定的格式化符號，可以將欲輸入的變數覆寫成已格式化的輸入源。

欲輸入：想被覆寫的變數。可以多個。在大部分情況下，欲輸入前面都需要加 **&** 來升等成「**可被覆寫**」的狀態。

以詳細的角度來說，其實並不是升等成「可被覆寫」，而是提供自己的記憶體位置，讓其他非本域的函數直接使用該變數。因此能修改、讀取。

輸入與輸出

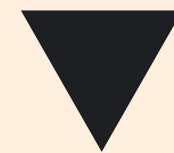
輸入

```
int age = 0;
```

```
scanf(format: "%d", &age);
```

使用者輸入

16



變數被覆寫

```
age = 16;
```